



Department of Computer Engineering

A Project Review 2 Presentation on

AION : Autonomous Incident Observation & Navigation

An AI-Powered Incident Observation and Remediation Platform using Log Analysis, Git Correlation and Large Language Models for Root-Cause Analysis, Validated Code Fixes and Human-Approved Deployment

Presented by :

Aditya Bheke – 330114

Harshal Bhogawade – 330115

Malhar Bhoi – 330116

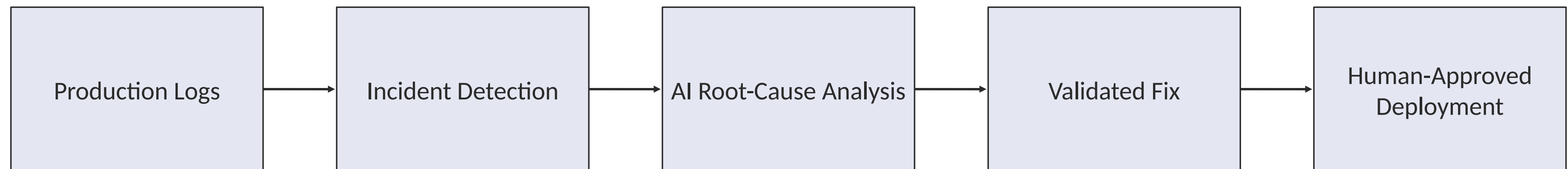
Under the guidance of : Dr. Yogesh Bahendwar

Introduction

Why AION?

- In conventional incident management, engineers manually inspect large volumes of logs during an outage.
- Tracing a failure to the recent code change that caused it is slow and depends on experienced engineers.
- Observability tools detect errors but stop there; writing, testing and deploying the fix remain separate manual steps.
- AION connects detection, Git correlation, AI root-cause analysis, patch generation and validation in one auditable workflow, with a human approving every production deployment.

Core Idea



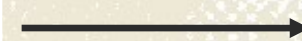
Problem Statement

Design and development of an AI-powered incident observation and remediation platform that collects and analyses production application logs, detects and deduplicates incidents, correlates them with Git commits and deployment history, performs evidence-based root-cause analysis using Large Language Models, generates candidate code patches that are validated through automated testing and staging, and deploys a fix to production only after explicit human approval.



Input

- Production application logs
- Git commit history
- Deployment records



AION

- Log analysis & incident detection
- Git & deployment correlation
- LLM root-cause analysis
- Patch generation & validation



Output

- Incident report & RCA
- Ranked suspect commits
- Validated code patch
- Human-approved, verified deployment

Literature Review

Sr. No.	Paper & Author’s Name	Publication	Description	Challenges
1	A Survey of AIOps in the Era of Large Language Models (L. Zhang, T. Jia, M. Jia, et al.) [12]	ACM Computing Surveys (2026)	Reviews 183 papers (2020–2024) on applying LLMs to AIOps: failure data sources, evolving tasks, LLM-based methods and evaluation.	• LLM limits in AIOps not yet understood • Legacy failure data hard to process • Immature evaluation of LLM-based AIOps
2	LLM4Log: A Systematic Review of Large Language Model-based Log Analysis (Z. Ma, J. Yang, T.-H. Chen) [13]	arXiv (2026)	Reviews 145 papers on LLM-based log analysis across seven tasks, from log parsing and anomaly detection to root-cause analysis.	• Log drift and long-tail events • Hallucination and unfaithful outputs • Context, cost, latency and privacy limits
3	Log Anomaly Detection in AIOps: A Real-World Implementation Using LLMs (M. De la Cruz Cabello, et al.) [14]	Systems and Soft Computing (2026)	Deploys a self-supervised LogBERT model, trained only on normal Linux syslogs and run locally, to detect anomalous log sequences in real time.	• Shifting log formats and conditions • Window size: accuracy vs. latency • Detection only: no root cause or fix
4	RepairAgent: An Autonomous, LLM-Based Agent for Program Repair (I. Bouzenia, P. Devanbu, M. Pradel) [8]	ICSE (2025)	An autonomous LLM agent that gathers bug information, searches for fixes and validates them with tools; repaired 164 Defects4J bugs.	• High LLM cost: ~270K tokens per bug • Benchmark bugs, not live incidents • Needs existing tests to validate fixes
5	Integrating LLMs with Cloud-Native Observability for Automated Root Cause Analysis and Remediation [15]	Intl. Conf. on AI, Systems and Network Security (2025)	Connects LLMs with cloud-native observability for automated root-cause analysis and remediation.	• Risk of hallucinated root causes • Telemetry volume exceeds LLM context • Auto-remediation needs safety checks

Research Gap

Our research studied existing work in AIOps, log analysis, LLM-based root-cause analysis and automated program repair, and tools such as ELK Stack, Sentry and PagerDuty. A consistent pattern emerged:



What Research Has Achieved

1. Parses and analyses large log volumes and detects anomalies automatically.
2. Uses LLMs and LLM agents to diagnose the root causes of cloud incidents.
3. Shows LLM agents can generate and validate code fixes for known bugs.



Where Research Stops

1. Treats detection or root-cause analysis as the final goal.
2. Repairs code on benchmark bugs, disconnected from live telemetry and CI/CD pipelines.
3. Tools stop at search, alerting or fix suggestions; validation and safe deployment stay manual.



What AION Does Differently

1. Connects detection, root-cause analysis, patch generation and validation in one workflow.
2. Correlates incidents with Git commits and deployment history to find the probable cause.
3. Validates every fix through tests and staging, and deploys only after human approval.

Motivation



Slow Incident Resolution

- Engineers inspect logs and recent changes by hand while users are affected.
- Resolution depends on experienced engineers being available.



Log and Alert Overload

- Thousands of repeated errors hide the one that matters.
- Deduplication and spike detection surface the real incident.



Fragmented Tooling

- Logs, Git, CI/CD and tickets live in separate tools.
- Context is lost at every hand-off between them.



Unsafe Automation

- A fully automatic fix can make production worse.
- AI output needs deterministic validation and a human approval gate.

Objective of the Project

01 Detection & Correlation

- Collect and normalize logs from file and HTTP sources.
- Group repeated errors into signatures; detect error spikes.
- Correlate incidents with Git commits and deployments.

02 AI Diagnosis & Documentation

- Evidence-based root-cause analysis with grounding checks.
- Structured incident report and ticket for every incident.
- Cloud (Claude API) or local LLMs for sensitive logs.

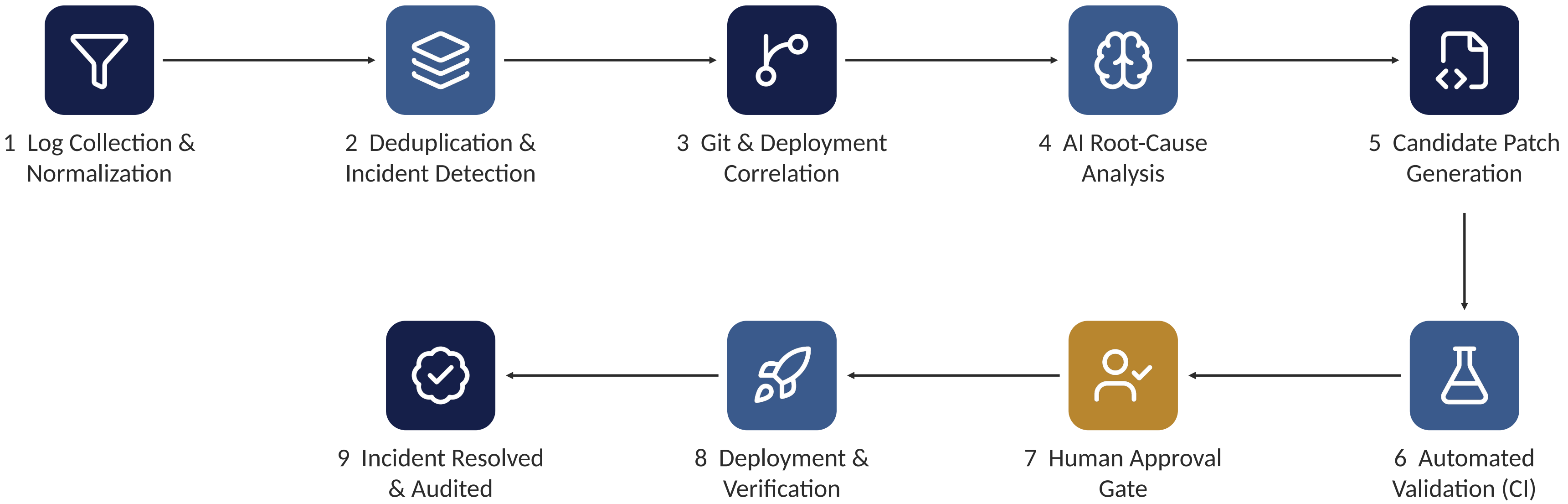
03 Validated Remediation

- Candidate patches as reviewable diffs on isolated branches.
- Regression tests that reproduce the incident.
- Validation: unit tests, staging deployment, request replay.

04 Safe Deployment & Monitoring

- Human approval gate bound to the exact validated commit.
- Deploy approved fixes, verify recovery, keep an audit trail.
- Integrated dashboard; evaluation on real bug scenarios.

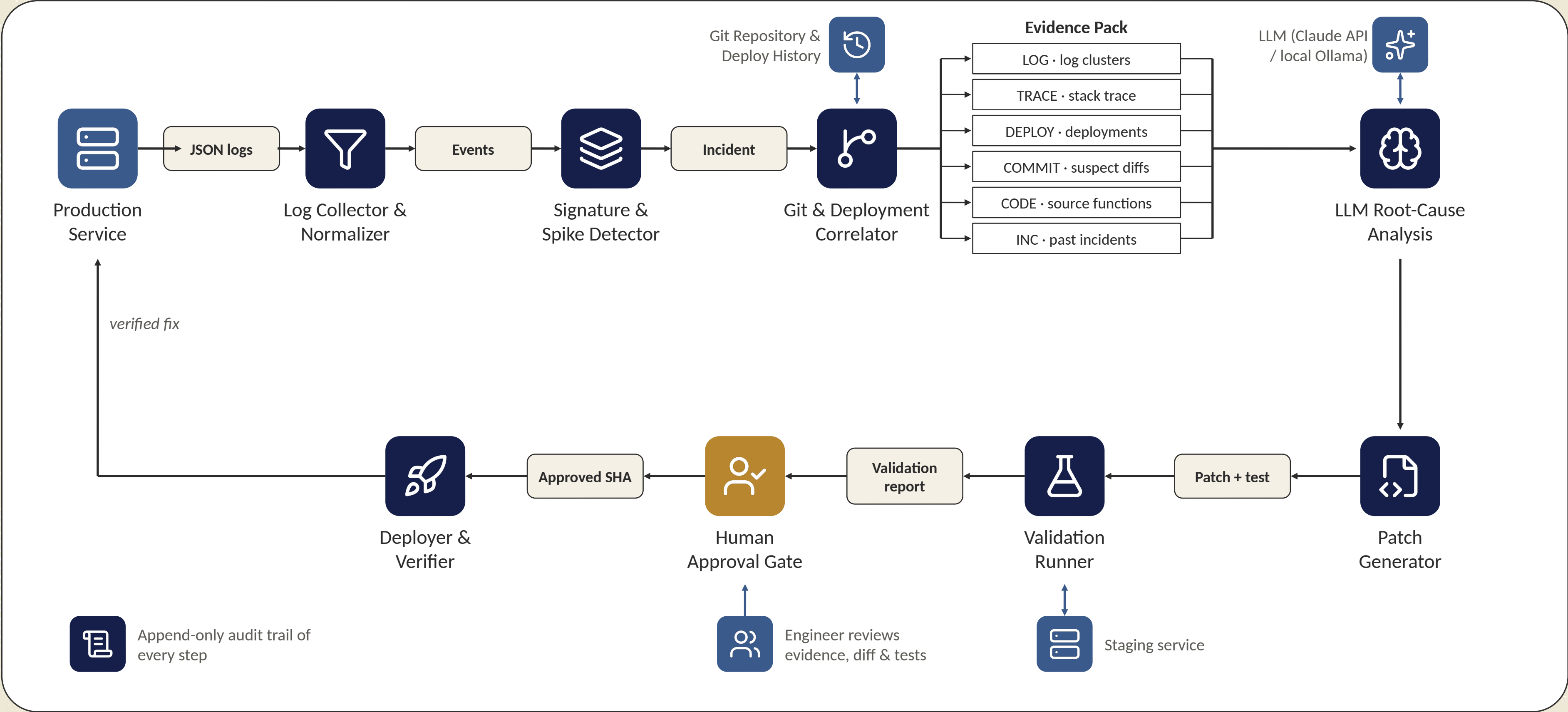
Methodology



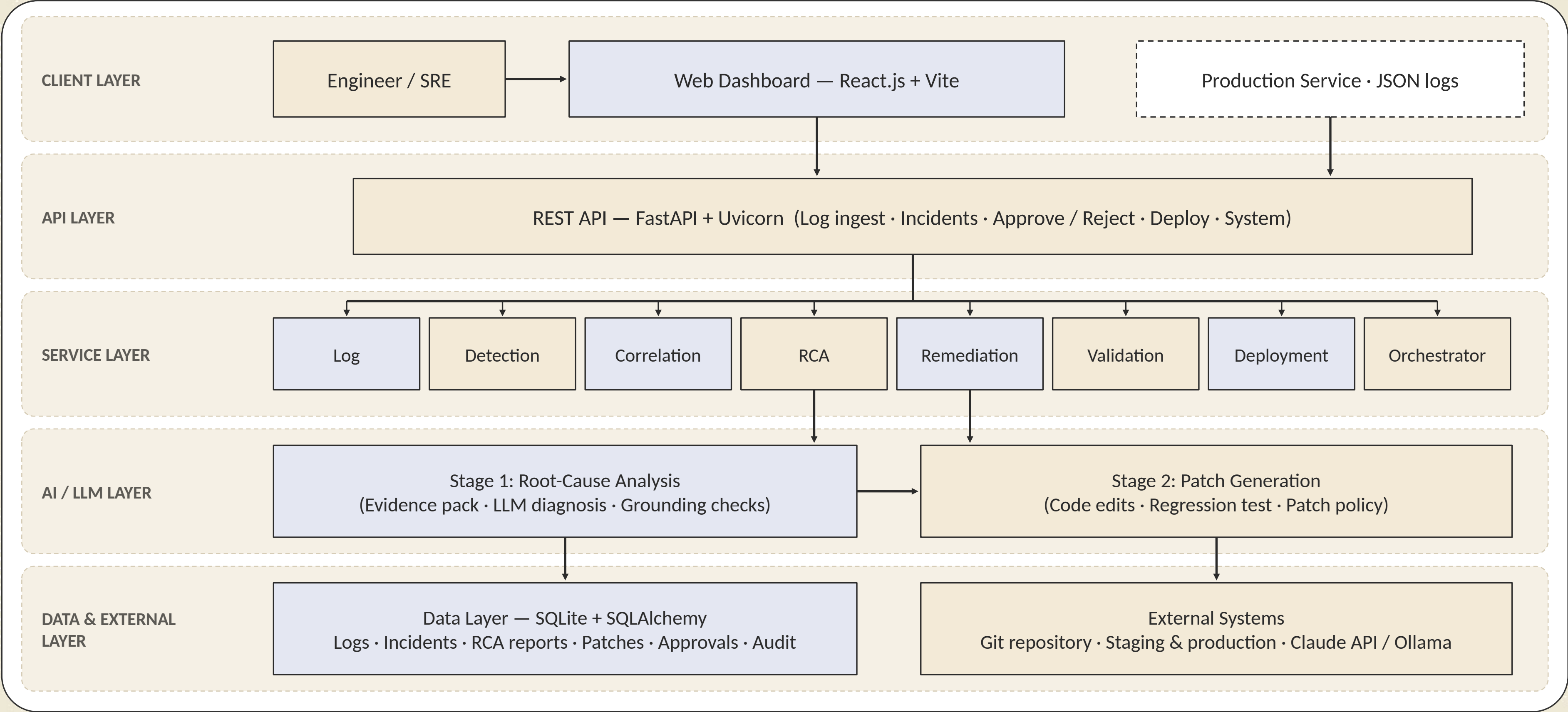
If validation fails: bounded repair loop, then git revert fallback

JSON logs → signatures → error spike → suspect commits → evidence-cited RCA → patch + test → CI & staging → approval → verified fix

System Architecture



Proposed System Architecture



System Modules



Observation & Analysis Modules

M1 Log Collection & Normalization: file tail, HTTP ingest, stack-frame parsing

M2 Deduplication & Incident Detection: signatures, spike vs. baseline

M3 Git & Deployment Correlation: git blame, deployment window, suspect ranking

M4 AI Root-Cause Analysis: evidence pack, LLM diagnosis, grounding checks

M5 Incident Report & Ticket: RCA document, cited evidence, issue ticket



Remediation & Control Modules

M6 Candidate Patch Generation: isolated branch, code edits + regression test

M7 Automated Validation: unit tests, fail-to-pass check, staging, replay

M8 Human Approval Gate: approval bound to the validated commit SHA

M9 Deployment & Verification: fast-forward merge, recovery check

M10 Dashboard & Audit Trail: pipeline view, diff viewer, audit log

Mathematical Model

Notation

L = stream of application log events

s = error signature (masked template + stack)

$c_t(s)$ = occurrences of s in current window

$b(s)$ = average occurrences per baseline window

C = recent commits, D = deployments

$S(c)$ = suspicion score of commit c

E = evidence pack, R = root-cause report

P = candidate patch, A = human approval

Formulation

Signature: $s = h(\text{type}, \text{frames}, \text{mask}(\text{message}))$

Incident: $c_t(s) \geq m$ and $c_t(s) \geq \rho \cdot b(s)$

Suspect score: $S(c) = \min(1, \sum w_k \cdot f_k(c)) \cdot \lambda(c)$

Probable cause: $c^* = \arg \max S(c)$

Validation: $V(P) = \text{syntax} \wedge \text{fail-to-pass} \wedge \text{tests} \wedge \text{staging} \wedge \text{replay}$

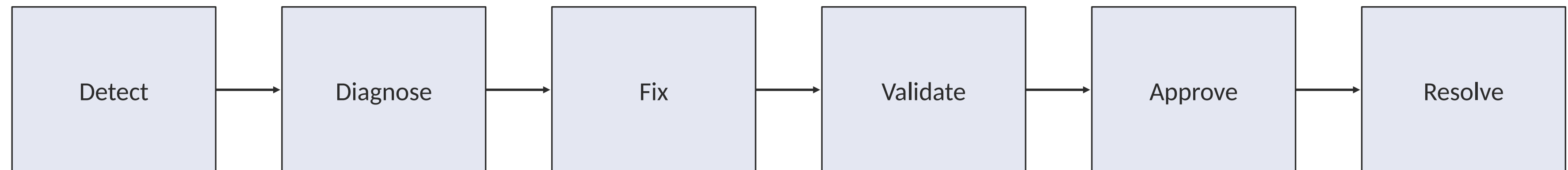
Deploy rule: $\text{deploy}(P) \Leftrightarrow V(P) = 1 \wedge A = \text{approve} \wedge \text{sha}(A) = \text{sha}(P)$

System output: $\text{AION}(L, C, D) \rightarrow \{ I, c^*, R, P, V, A \}$

Defaults: $m = 5$ errors, $\rho = 3$, window 60 s, baseline 30 min. Weights w : blame line 0.40, blame function 0.20, deployment 0.20, file 0.15, dependency 0.15, keyword 0.05; $\lambda = 0.5$ for code already stable before the incident. $S(c)$ ranks suspects; it is not a probability.

Conclusion

- AION combines log analysis, Git correlation and LLM-based root-cause analysis in one incident workflow.
- It turns an error spike into an evidence-cited diagnosis with a ranked list of suspect commits.
- It generates candidate fixes with regression tests and validates them through tests, staging and request replay.
- Only a human can approve production deployment, and every step is recorded in an audit trail, reducing resolution time without unsafe automation.



References

- [1] P. Notaro, J. Cardoso, and M. Gerndt, “**A Survey of AIOps Methods for Failure Management,**” ACM Transactions on Intelligent Systems and Technology, vol. 12, no. 6, 2021.
- [2] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, “**Drain: An Online Log Parsing Approach with Fixed Depth Tree,**” in Proc. IEEE ICWS, 2017, pp. 33–40.
- [3] M. Du, F. Li, G. Zheng, and V. Srikumar, “**DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning,**” in Proc. ACM CCS, 2017, pp. 1285–1298.
- [4] W. Meng et al., “**LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs,**” in Proc. IJCAI, 2019, pp. 4739–4745.
- [5] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, “**MicroRCA: Root Cause Localization of Performance Issues in Microservices,**” in Proc. IEEE/IFIP NOMS, 2020.
- [6] T. Wittkopp, A. Acker, and O. Kao, “**Progressing from Anomaly Detection to Automated Log Labeling and Pioneering Root Cause Analysis,**” AIOps Workshop, IEEE ICDM, 2023.
- [7] Y. Chen et al., “**Automatic Root Cause Analysis via Large Language Models for Cloud Incidents,**” in Proc. EuroSys, 2024.
- [8] I. Bouzenia, P. Devanbu, and M. Pradel, “**RepairAgent: An Autonomous, LLM-Based Agent for Program Repair,**” in Proc. IEEE/ACM ICSE, 2025.

References (contd.)

- [9] Q. Zhang et al., **“A Systematic Literature Review on Large Language Models for Automated Program Repair,”** arXiv:2405.01466, 2024.
- [10] D. Roy et al., **“Exploring LLM-based Agents for Root Cause Analysis,”** in Proc. ACM FSE, 2024.
- [11] U. Kulsum, H. Zhu, B. Xu, and M. d’Amorim, **“A Case Study of LLM for Automated Vulnerability Repair: Assessing Impact of Reasoning and Patch Validation Feedback,”** in Proc. ACM AIware, 2024.
- [12] L. Zhang, T. Jia, M. Jia, et al., **“A Survey of AIOps in the Era of Large Language Models,”** ACM Computing Surveys, 2026.
- [13] Z. Ma, J. Yang, and T.-H. Chen, **“LLM4Log: A Systematic Review of Large Language Model-based Log Analysis,”** arXiv:2604.16359, 2026.
- [14] M. De la Cruz Cabello, T. P. Sales, and M. R. Machado, **“Log Anomaly Detection in AIOps: A Real-World Implementation Using Large Language Models,”** Systems and Soft Computing, vol. 8, 2026.
- [15] **“Integrating Large Language Models with Cloud-Native Observability for Automated Root Cause Analysis and Remediation,”** Intl. Conf. on AI, Systems and Network Security, 2025.